Tema 7 — LTS y lógica temporal (LTL)

Fuentes: teórica 09-LTS-logicas (FSP, composición, deadlock; termina en la definición de LTL), apunte de la práctica 7 ("Modelos y lógica para modelar concurrencia", **el más útil, leelo entero**), Guía 7. **En el parcial no entra teoría directa: solo ejercicios tipo guía.**

---

0. Notación de la cátedra

Símbolo	Otros nombres	Se lee	Definición (σ, i ⊨ ...)
`○ϕ`	`X ϕ`, next	"en el siguiente"	φ vale en i+1
`◇ϕ`	`◊ϕ`, `<>ϕ`, `F ϕ`	"eventualmente"	existe j **≥ i** con φ
`□ϕ`	`[]ϕ`, `G ϕ`	"siempre"	para todo j **≥ i** vale φ
`ϕ U ψ`	until	"φ hasta ψ"	existe k ≥ i con ψ, y φ en todas las j con i ≤ j < k. **ψ tiene que llegar**
`ϕ W ψ`	weak until, "awaits"	"φ hasta ψ, si llega"	`(ϕ U ψ) ∨ □ϕ`

- **Trazas:** sucesiones infinitas de conjuntos de proposiciones, numeradas **desde 1**: `σ = {p} {p,q} ∅ ({q})ω`. `wω` = repetir w infinitas veces; la ω va **siempre al final**.
- `σ, i ⊨ ϕ` = φ vale **mirando la traza desde i en adelante** (no solo la posición i). `σ ⊨ ϕ` sii `σ, 1 ⊨ ϕ`.
- La guía usa **solo LTL** (no CTL). CTL aparece en el apunte como "para saber" (sección 9).
- Conectivos: `¬ ∧ ∨ →`, evaluados en la misma posición.

---

1. LTS: qué es y cómo se compone

Un **LTS** es `(S, A, →, s0)`: estados, alfabeto de acciones, transiciones `s –a→ s'`, estado inicial. - **Lo que no está en el LTS no puede pasar.** Si no hay flecha `acquire` desde `tomado`, no se puede hacer. - Varias salidas desde un estado = **no determinismo = el scheduler elige**. - **Estado sin salidas** = deadlock **o** terminación. El LTS no los distingue: lo decidís vos (¿todos los threads en `fin`? terminación; si no, deadlock). - Solo dibujamos estados **alcanzables**.

1.1 Composición paralela `P ‖ Q` ⭐

Estados = pares `(p, q)`, alfabeto = `A_P ∪ A_Q`, inicial = `(p0, q0)`. Cada acción cae en uno de tres grupos:

Grupo	Regla	Efecto
propia de P (`A_P \ A_Q`)	`p –a→ p'` ⇒ `(p,q) –a→ (p',q)`	**interleaving**: avanza P, Q quieto
propia de Q (`A_Q \ A_P`)	simétrica	avanza Q, P quieto
compartida (`A_P ∩ A_Q`)	solo si **los dos la ofrecen**: `(p,q) –a→ (p',q')`	**sincronización**: avanzan juntos en un paso

- **El grupo lo decide el alfabeto, no el dibujo.** Si `a ∈ A_P` pero P nunca la ofrece, en la composición nadie la puede hacer.
- Con más componentes (`P ‖ Q ‖ R`, sin paréntesis): una acción la hacen **juntos todos los que la tienen en el alfabeto**; si alguno de ellos no la ofrece, no pasa. Cada componente **bloquea** acciones de su alfabeto con solo no ofrecerlas.
- **Construcción:** arrancás en el par inicial, aplicás las reglas, repetís con cada estado nuevo hasta que no aparezcan más (BFS). No armes el producto entero.

> **En criollo:** las acciones propias se intercalan como quiera el scheduler; las compartidas son un apretón de manos: si uno no extiende la mano, el otro se queda esperando.

1.2 La trampa del nombre compartido ⭐

Si T1, T2 y LOCK usan todos `acquire`, la composición obliga a los **tres** a hacerlo juntos → los dos threads entran a la vez (`(N,N,libre) –acquire→ (C,C,tomado)`). **El modelo está mal, no el lock. Arreglo:** la acción dice quién la hace: `acquire.i`, `release.i`. El thread i solo tiene las suyas; el recurso (lock, buffer, variable) ofrece las de todos. (En FSP de la teórica: `a:USER || b:USER || {a,b}::RESOURCE`.)

---

2. Modelar un programa como LTS (receta) ⭐

Convención de la materia: - **Memoria compartida** → estado = **pc de cada thread + variables**; transiciones etiquetadas con la línea ejecutada. - **Pasaje de mensajes / composición** → por **acciones**, cada cosa (thread, canal, lock) es un LTS y se componen.

2.1 Variables compartidas, en 4 pasos

1. **¿Qué es un estado?** Una foto con todo lo que hace falta para decidir el futuro: `(pc_1, pc_2, ..., variables)`. Pregunta clave: *si dos situaciones se pueden comportar distinto en el futuro, son estados distintos.* Ojo con las **locales** que se leen después (en `incr()`, `loc` importa porque se usa en la línea siguiente).
2. **Diagrama de control de cada thread solo:** por qué líneas pasa, qué efecto tiene cada una, de qué depende cada bifurcación (`if`, `while`, `await`).
3. **Regla para avanzar:** desde un estado, cualquier thread no terminado (y no bloqueado) ejecuta su próxima línea: cambia **su** pc y las variables que escribe; el resto queda igual.
4. **Construcción:** desde el inicial, aplicar la regla a cada estado nuevo. Marcar los estados interesantes (los "malos", los sin salida).

Instrucciones típicas:

Instrucción	En el LTS
línea atómica `x = e`	una transición que cambia pc y x
`if (c)` / `while (c)`	dos salidas posibles según el valor **en el estado**; leer no cambia variables
`await (c)`	transición habilitada **solo si c vale** en el estado; si no, ese thread no tiene salida ahí (está bloqueado)
asignación múltiple atómica `(a, b) = (e1, e2)`	**una sola** transición que cambia las dos
`lock()`	habilitada solo si el lock está libre (o modelá el lock como LTS aparte con `acquire.i`)
`tryLock()`	**nunca bloquea**: dos salidas, `true` (si libre: lo toma) o `false` (si tomado), como un `if`
`loop { ... }`	el último pc vuelve al primero (el LTS tiene ciclos)
fin del thread	pc = `fin`, sin salidas para ese thread

2.2 Achicar el modelo (sin romperlo)

- Lo que **no influye en el futuro** ni en la pregunta, o **se deduce de otra cosa**, se saca (ej. `open` se deduce de `pc_closer`; `sold` nunca se lee).
- Líneas que **no tocan nada compartido** se pueden pegar a la siguiente.
- **Nunca** pegues dos líneas que leen/escriben algo compartido: entre ellas puede meterse el otro thread (ahí vive el bug "chequeo y después actúo").
- Qué va en el estado **depende de lo que quieras preguntar**.

2.3 Canales (Go) ⭐

Canal	Modelo
sin buffer `make(chan T)`	mandar y recibir son **una sola acción compartida** entre emisor y receptor: solo pasa si uno está mandando **y** el otro recibiendo a la vez
con buffer `make(chan T, 1)`	el canal es **un LTS más** (`vacío` / `lleno`, o `lleno(v)` si importa el valor). Enviar = `in.i` (solo si vacío), recibir = `out.i` (solo si lleno). Ya no ocurren a la vez
varios emisores en el mismo canal	acciones con nombre de quién: `in.1`, `in.2` (trampa 1.2)
importa **qué valor** viaja	acciones indexadas por valor (`send.v`, `recv.v`) y el buffer recuerda el valor

Ejemplo del apunte: `mu := make(chan bool, 1)` con `mu <- true; critical(); <-mu` es **un lock** (mismo LTS de 3 estados). Con `make(chan bool)` sin buffer: todos quieren mandar, nadie recibe → **deadlock desde el inicio**.

2.4 Variable como proceso (estilo teórica)

También se puede modelar `seat` como un LTS `SEAT` con un estado por valor: `p.read.v` (solo la ofrece en el estado v, lazo), `p.write.v` (siempre, pasa a v). El `if` del thread se vuelve sincronización: el thread ofrece todas las lecturas y SEAT bloquea las que no corresponden. Da el **mismo grafo**. Para LTL conviene la versión con proposiciones en los estados.

2.5 Deadlock en el LTS

- **Deadlock** = estado alcanzable sin salidas que no es la terminación correcta.
- **Camino más corto a deadlock:** BFS desde el inicial (es lo que hace LTSA).
- Para **mostrar que no hay deadlock**: recorrer todos los estados alcanzables y ver que cada uno (salvo el final legítimo) tiene salida.
- Arreglos clásicos (teórica): **orden global de adquisición** (todos toman los recursos en el mismo orden), **rendirse y soltar** (timeout / `tryLock`: si no consigo el segundo, suelto el primero), **romper la simetría** (filósofos pares/impares), watchdog que aborta.

---

3. LTL sobre trazas

3.1 Cómo evaluar una fórmula en una traza (técnica central) ⭐

Como una tabla de verdad: **una fila por subfórmula, una columna por posición**, de **adentro hacia afuera**. 1. Escribí qué vale en cada posición (y cuál es el patrón periódico del final). 2. Marcá dónde valen las proposiciones. 3. Subí: primero `◇q`, después `p → ◇q`, recién al final `□(p → ◇q)`. 4. La respuesta es lo que vale en la **posición 1**.

Atajos por operador:

Fórmula	Cómo chequearla en σ
`○ϕ`	φ en la posición 2
`○○ϕ`	φ en la posición 3
`◇ϕ`	¿aparece φ en algún lado (incluida la 1)?
`□ϕ`	¿φ en **todas**?
`□◇ϕ`	¿φ aparece en la **parte periódica** (la de la ω)?
`◇□ϕ`	¿φ vale en **toda** la parte periódica?
`p U q`	buscá la **primera** q; p tiene que valer en todas las anteriores (en la de q no hace falta). Si no hay q → falso
`p W q`	igual que U, pero si q no llega alcanza con □p
`□(p → ◇q)`	para cada p, ¿hay q en esa posición o después? (con ω: mirá cada p de la parte periódica)
`□(p U q)`	p U q en **todas** las posiciones: q tiene que seguir apareciendo para siempre, y entre q y q, p

3.2 Frases en castellano → fórmula ⭐⭐

Castellano	LTL	Tipo
siempre p / p es invariante	`□p`	safety
nunca p	`□¬p`	safety
p y q nunca a la vez	`□¬(p ∧ q)`	safety
eventualmente p / alguna vez p	`◇p`	liveness
en el próximo paso p	`○p`	—
cada vez que p, en el paso siguiente q	`□(p → ○q)`	safety
infinitas veces p / "siempre, eventualmente p"	`□◇p`	liveness
a partir de algún momento, siempre p / p se estabiliza	`◇□p`	liveness
p ocurre finitas veces	`◇□¬p` (≡ `¬□◇p`)	liveness
p no dura para siempre (cada vez que vale, en algún momento deja de valer)	`□(p → ◇¬p)` (≡ `¬◇□p`)	liveness
si p, entonces eventualmente q (respuesta)	`□(p → ◇q)`	liveness
una vez que p, q para siempre	`□(p → □q)`	safety
p hasta que q (y q llega)	`p U q`	safety + liveness
p hasta que q, si es que q llega	`p W q`	safety
q no pasa antes de p (precedencia: "si q, antes p")	`¬q W p`	safety
después de p, q no vale hasta r	`□(p → (¬q W r))`	safety
si p infinitas veces, entonces q infinitas veces	`□◇p → □◇q`	liveness
si a partir de algún momento p siempre, entonces q infinitas veces	`◇□p → □◇q`	liveness
desde el inicio, p hasta que q, después r siempre	`p U (q ∧ □r)`	—

Lectura: **de afuera hacia adentro**. `□(...)` = "en todo momento ...", `◇(...)` = "en algún momento ...". "**Cada vez que**" = `□(... → ...)`. "**De ahí en adelante**" = otro `□` adentro.

> **En criollo:** `□` es un "para todo momento futuro", `◇` es un "existe un momento futuro". `□◇` = "no deja nunca de volver a pasar". `◇□` = "en algún momento se queda pegado".

3.3 Propiedades típicas de concurrencia ⭐

Con proposiciones de estado `wait_i` / `try_i` (el thread i quiere entrar), `critical_i`:

Propiedad	Fórmula	Tipo
Exclusión mutua	`□¬(critical_1 ∧ critical_2)`	safety
Ausencia de inanición (thread i)	`□(wait_i → ◇critical_i)`	liveness
Ausencia de deadlock (progreso global: si alguien quiere, alguien entra)	`□((wait_1 ∨ wait_2) → ◇(critical_1 ∨ critical_2))`	liveness
Entra infinitas veces	`□◇critical_i`	liveness
No entra sin permiso	`¬critical_i W acquired_i`	safety

(Las definiciones exactas de las proposiciones las da cada ejercicio: leé **cuándo vale** cada una, por ejemplo en el Ej 7 `request_i` vale desde que llama a `lock()` hasta que libera, y `release_i` solo en el instante del `unlock()`.)

3.4 Safety vs liveness ⭐

Pregunta: si se viola, ¿puedo señalar el momento en que se violó? - **Sí** → **safety** ("nunca pasa algo malo"). Contraejemplo = **prefijo finito** (camino del inicial hasta lo malo). Ej: `{p} ∅` ya refuta `□p`. - **No, hay que mirar la ejecución entera** → **liveness** ("algo bueno termina pasando"). Ningún prefijo finito la refuta. Contraejemplo = **lazo** (infinito). - Mixtas: `ϕ U ψ ≡ (ϕ W ψ) ∧ ◇ψ` → parte safety + parte liveness; tratalas por separado.

3.5 Leyes útiles (para equivalencias y simplificar)

Ley	
Dualidad	`¬○ϕ ≡ ○¬ϕ`, `¬◇ϕ ≡ □¬ϕ`, `¬□ϕ ≡ ◇¬ϕ`
Distribuye	`□(ϕ ∧ ψ) ≡ □ϕ ∧ □ψ`, `◇(ϕ ∨ ψ) ≡ ◇ϕ ∨ ◇ψ`
No distribuye	`□` sobre `∨` (solo `□ϕ ∨ □ψ → □(ϕ ∨ ψ)`), `◇` sobre `∧` (solo `◇(ϕ∧ψ) → ◇ϕ ∧ ◇ψ`)
`○` distribuye sobre todo y **conmuta**	`○□ϕ ≡ □○ϕ`, `○◇ϕ ≡ ◇○ϕ`, `○(ϕ U ψ) ≡ (○ϕ) U (○ψ)`
Absorción	`□□ ≡ □`, `◇◇ ≡ ◇`, `◇□◇ ≡ □◇`, `□◇□ ≡ ◇□` (toda tira se reduce a `□`, `◇`, `□◇`, `◇□`)
Expansión	`◇ϕ ≡ ϕ ∨ ○◇ϕ`, `□ϕ ≡ ϕ ∧ ○□ϕ`, `ϕ U ψ ≡ ψ ∨ (ϕ ∧ ○(ϕ U ψ))`
Definiciones cruzadas	`◇ϕ ≡ true U ϕ`, `□ϕ ≡ ¬◇¬ϕ`, `□ϕ ≡ ϕ W false`, `ϕ W ψ ≡ (ϕ U ψ) ∨ □ϕ`, `ϕ U ψ ≡ (ϕ W ψ) ∧ ◇ψ`
Negar un U	`¬(ϕ U ψ) ≡ ¬ψ W (¬ϕ ∧ ¬ψ)`
Relación	`◇□ϕ → □◇ϕ` vale, al revés no
Implicación	`ϕ → ψ ≡ ¬ϕ ∨ ψ` (útil para pasar `□ϕ → ◇ψ` a la forma `◇χ`: `◇¬ϕ ∨ ◇ψ ≡ ◇(¬ϕ ∨ ψ)`)

3.6 Demostrar equivalencias / validez

- **Equivalencia `ϕ ≡ ψ`:** para cualquier σ y **posición genérica i**: si vale una vale la otra (⇒ y ⇐). Trabajá con la definición como cuantificadores ("existe j ≥ i", "para todo j ≥ i").

- **Refutar:** una traza concreta (lo más chica posible, tipo `({p} {q})ω`, `∅ ({p})ω`) donde un lado vale y el otro no. Probá con trazas que **alternan** y con trazas que **cambian una vez y se quedan**.
- **Satisfacible:** existe una traza que la cumple (mostrala). **Válida:** todas la cumplen (demostrá) — si no, mostrá una que no. **Insatisfacible:** ninguna (ej. `□p ∧ ◇¬p`).
- Para una implicación `A → B`: suponé que A vale en 1 (si no, vale trivial) y probá B. `□(p → ○p) → (p → □p)` es **inducción** en LTL.
- Si hay que razonar "la primera posición donde vale χ", usá U/W (sugerencia del Ej 5e).

---

4. LTL sobre un LTS

4.1 De ejecuciones a trazas

- A cada estado le asignás `L(s)` = proposiciones que valen ahí (calculadas desde el pc y las variables, o el nombre del estado).
- Ejecución `s1 s2 s3 ...` ⇒ traza `L(s1) L(s2) L(s3) ...`.
- **Un LTS satisface φ si TODAS sus ejecuciones desde el inicial la satisfacen.** Refutar: una ejecución. Demostrar: razonar sobre todas.
- **Convención:** una ejecución que llega a un estado sin salidas **se queda ahí para siempre** (el último estado se repite: `... (s_final)ω`). Por eso un programa que termina satisface `◇□(lo que vale al final)`.
- Usamos **proposiciones sobre estados**, no sobre acciones (con acciones, "los dos decidieron escribir" no vale en ninguna posición; hay que describir secuencias). Excepción: el Ej 11 pide proposiciones = acciones (en cada posición vale solo la acción de ese paso).

4.2 Receta para verificar una fórmula en un LTS ☆☆

1. **Clasificá** la fórmula: safety o liveness (3.4).
2. **Safety** (`□¬bad`, `□(p → □q)`, `W`): - **Vale** si en ningún estado alcanzable pasa lo malo (recorrés la lista de estados). Para `□(p → □q)`: desde todo estado con p, todos los alcanzables tienen q. - **No vale:** contraejemplo = **camino finito** desde el inicial hasta el estado malo. Lo que sigue no importa.
3. **Liveness** (`◇p`, `□◇p`, `□(p → ◇q)`): - **No vale** si hay un **lazo malo**: un tramo desde el inicial (pasando por p, si hay antecedente) hasta un estado s, y una **vuelta** de s a s que **nunca pasa por lo bueno**. Se escribe `s0 s1 (s2 s3 s4)ω`. **Un estado sin salidas cuenta como lazo de largo 1.** - **Vale** si no existe tal lazo. Para `□(wait → ◇crit)`: mirá el subgrafo de estados con `wait` y sin `crit` alcanzables desde un `wait`; si **no tiene ciclos ni estados sin salida**, vale. Para `□◇p`: todo ciclo alcanzable y todo estado sin salida tiene algún p. - ¿Por qué alcanzan lazos? El LTS es finito: una ejecución infinita que evita lo bueno repite algún estado.
4. **¿Y bajo fair choice?** (4.3).
5. Reportá: "vale" con el argumento, o "no vale" con el contraejemplo **escrito como secuencia de estados** (y si es lazo, con la ω).

4.3 Fair choice y componentes terminales ☆

Fair choice: una ejecución es válida si, **para cada estado que visita infinitas veces, toma infinitas veces cada una de sus transiciones salientes**. Modela un scheduler que no discrimina para siempre.

Componente terminal: conjunto de estados donde desde cualquiera se llega a cualquier otro, y **del que no sale ninguna transición**. (Un estado sin salidas es una componente terminal de un estado.)

Resultado clave: **toda ejecución justa termina metida en una componente terminal y la recorre entera, infinitas veces.**

Reglas prácticas: | Fórmula | Bajo fair choice vale sii... | |---|---| | `□◇p` | **toda** componente terminal alcanzable tiene algún estado con p | | `◇□p` | toda componente terminal alcanzable tiene p en **todos** sus estados | | `□(p → ◇q)` | desde cada estado con p, no se puede llegar **sin pasar por q** a una componente terminal sin q | | safety | **igual que sin fairness**: un prefijo malo se puede continuar de forma justa. Fair choice **no salva safety** |

- Un lazo es **injusto** si su vuelta pasa por un estado y nunca toma una de sus salidas → no cuenta como contraejemplo bajo fair choice.
- Un estado **sin salidas** (deadlock o thread que terminó y dejó al otro esperando) **no lo arregla fair choice**: no hay transiciones que obligar. El contraejemplo es del protocolo, no del scheduler.
- "Tardar todo lo que quiera" (lazo `trabaja` en N) ≠ "terminar": bajo fair choice tardar no es contraejemplo, terminar sí.

Receta "identificá los conjuntos terminales": (1) buscá los ciclos de estados que se alcanzan mutuamente (componentes fuertemente conexas); (2) quedate con las que **no tienen flechas hacia afuera**; (3) para cada fórmula de liveness, mirá qué proposiciones aparecen en cada una.

---

5. Observadores (LTS de propiedad) — Ej 11

Una propiedad de **safety sobre acciones** se puede dar como un LTS "observador" que lee las acciones del sistema: 1. Estados = lo que hay que recordar de la historia (ej. "tengo el recurso / no lo tengo"). 2. Desde **cada** estado, una transición para **cada** acción del alfabeto: si la acción es legal, va al estado que corresponda; si viola la propiedad, va a un estado **ERROR** (en FSP, estado `-1` / π). 3. La propiedad se viola sii el sistema compuesto con el observador puede llegar a ERROR → contraejemplo = camino finito (es safety).

(Esto no está desarrollado en la teórica ni en el apunte; es el `property` de FSP/LTSA.)

---

6. Trampas (las que más importan)

1. `◇` **y** `□` **incluyen la posición actual.** `◇p` vale si p vale ya.
2. `U` **exige que ψ llegue**; si no llega es falso aunque φ valga siempre. Si no querés exigir la llegada, `W`.
3. **Implicación vacía:** `□(q → ◇p)` vale en una traza donde q no vale nunca. Y `p → ◇q` **sin** `□` solo mira la posición 1.
4. `□◇` **≠** `◇□`**.** `({p} ∅)ω` cumple `□◇p` pero no `◇□p`.
5. `□` **no distribuye sobre** `∨` ni `◇` **sobre** `∧`: contraejemplos típicos `({p}{q})ω` y `{p}{q}∅ω`.
6. **"Termina reservado" no es** `◇¬free` (eso es "alguna vez"); es `◇□¬free`.
7. `○` **y la composición:** si componés con otro thread que da pasos propios, entre dos pasos de tu thread se intercalan pasos del otro en los que tus proposiciones no cambian ("stuttering"). Las fórmulas con `○` que hablan de **un** thread se rompen; usá `◇`, `U`, `W` (idea del Ej 9).
8. **Nombre de acción compartido entre threads** = los obliga a hacerla juntos. Usá `acción.i`.
9. **Pegar líneas que tocan memoria compartida** esconde justo el bug.
10. **Olvidar variables locales que se usan después** (o el valor que viaja por un canal) en el estado.
11. **Canal sin buffer ≠ con buffer:** sin buffer es una acción compartida (el emisor espera al receptor); con buffer el envío no espera mientras haya lugar.

12. **Contraejemplo de liveness con un prefijo finito**: no existe; tiene que ser un lazo (o terminar en un estado sin salidas, que se repite).
13. **Fair choice no arregla safety ni estados sin salidas.**
14. **Deadlock vs terminación:** un estado sin salidas con todos en `fin` no es deadlock.
15. **Tipo de propiedad mal clasificado:** `□(p → □q)` es safety aunque tenga dos `□`; `□(p → ◇q)` es liveness.

---

7. Plantilla mental por tipo de ejercicio

"Dibujá el LTS de este programa" → receta 2.1: definí el estado, diagrama de control por thread, regla de avance, BFS. Marcá estados malos y sin salida.

"Modelá como composición de LTS" → un LTS por thread/canal/lock, acciones con nombre de quién (`x.i`), decidí qué es compartido, componé desde el inicial (1.1).

"¿Hay deadlock? Camino más corto. Arreglalo" → BFS hasta un estado sin salida que no es final. Arreglos: orden de adquisición, rendirse/soltar (`tryLock`). Mostrar que el nuevo LTS no tiene estados sin salida (salvo el final).

"¿Qué trazas satisfacen...?" → tabla posición × subfórmula, de adentro hacia afuera, mirar posición 1 (3.1).

"Equivalencia / satisfacible / válida" → definición con cuantificadores en posición i genérica; contraejemplo con traza chica (3.6).

"Escribí en LTL y clasificá" → tabla 3.2/3.3, leer bien cuándo vale cada proposición, clasificar con la pregunta de 3.4.

"Definí este operador" → expresalo con `U` / `W` / `□` / `◇` y probalo contra los casos borde: ¿qué pasa si ψ no llega nunca? ¿y si φ y ψ valen en la misma posición?

"Verificá sobre el LTS (con y sin fair choice)" → receta 4.2 + 4.3. Contraejemplo: camino (safety) o lazo con ω (liveness).

---

8. CTL (solo para saber; la guía no lo usa)

- LTL mira **cada ejecución por separado**: un sistema = su conjunto de trazas. No puede decir "**siempre es posible** conseguir el lock" (dos LTS con las mismas trazas, uno con un estado muerto `D`, satisfacen las mismas fórmulas LTL).
- **CTL** agrega cuantificadores de caminos: `A` (todos los caminos), `E` (existe un camino), siempre pegados a un operador temporal: `AG EF acquired`.
- Ninguna contiene a la otra: CTL no expresa `◇□p`. CTL* contiene a las dos. Bisimulación = equivalencia que ve la ramificación (más fina que igualdad de trazas).

---

9. Patrones para la Guía 7

Ej	Tipo	Técnica / idea
1	LTS de memoria compartida	estado `(pc_1, pc_2, loc_1, loc_2, x)`; las locales importan; buscá el estado final con la actualización perdida
2	composición con locks + deadlock	locks como LTS con `lock.i` / `unlock.i`; BFS al deadlock (cada uno con un lock esperando el otro); arreglos: `tryLock` y soltar / mismo orden de adquisición; mostrar que todo estado no final tiene salida
3	canales Go	sin buffer: envío = acción compartida con la recepción; con buffer 1: canal como LTS `vacío` / `lleno`; mirá qué hace cada uno primero
4	evaluar fórmulas en trazas	tabla posición × subfórmula; `□◇` / `◇□` se deciden en la parte periódica; `U`: primera q
5	equivalencias	prueba en posición i genérica / traza contraejemplo; leyes de 3.5 (distribución, `○` conmuta); 5e: pasar a `◇χ` y usar la primera posición de χ
6	satisfacible / válida	testigo para satisfacible; prueba con cuantificadores o contraejemplo para válida; ojo con implicaciones vacías
7	escribir propiedades + clasificar	tablas 3.2 y 3.3; respetar la semántica de `request_i` / `release_i`; (e) `□◇ → □◇`, (f) `◇□ → □◇`; (g) "a lo sumo una vez" = encadenar `W` / `U` contando entradas mientras dura la espera
8	definir operadores	con `U` / `W`; pensar el caso "ψ nunca vale" (¿es vacuo?) y la posición donde coinciden; B es una precedencia (`¬q W p`, ver 3.2)
9	robustez ante composición	el `○` se rompe con pasos ajenos (stuttering); reescribir con `◇` / `U` / `W`
10	verificar en LTS dado + fair choice	lazos malos (`TRYING ⇄ BACKOFF`, `giveup`); componentes terminales; safety vs liveness; regla de 4.3
11	observador + LTL sobre acciones	sección 5 (estado ERROR); en LTL: precedencia con `W` (`use` no antes de `acquire`, entre dos `acquire` un `release`)
12	LTS + mutex/inanición (algoritmo tipo Peterson)	estado `(pc_0, pc_1, y[0], y[1], s)`, asignación múltiple = una transición, `await` = transición condicionada; verificar con 4.2/4.3
13	LTS infinito (algoritmo tipo panadería)	acotar `y1, y2 ≤ 2`; para (b) una ejecución en la que los números crecen sin cota; mutex / deadlock / inanición con 3.3 y 4.2
14	canales con valores + propiedades	acciones/estados indexados por el valor (quién pidió); `responses` sin y con buffer; proposiciones de estado con el valor recibido; verificar con y sin fair choice

Estrategia general: primero el LTS (bien hecho, con la información justa en el estado), después las proposiciones de estado, después la fórmula, después clasificar (safety/liveness), y recién ahí buscar camino o lazo.